TWIN-64
Architecture Document

Helmut Fieres
July 20, 2025

Contents

Introduction

Concepts

This book

Architecture Overview

64 bit architecture

only signed 64 bit math

multi processor

processors have a TLB and a cache

52 bit virtual memory

34 bit physical memory

I/O is memory mapped

I/O Modules - System Bus

instructions feature register memory and load / store concept

register offset and register indexed, scalable

System Organization

Data Types

64 bit signed is the basic unit

lower sizes are sign extended before use

address computation uses a 32-bit unsigned math, i.e. numbers wrap around in 32 bit space.

Byte Ordering

General Registers

Control Registers

Processor Status Register

TLB

fully associative TLB

support for large pages (16Kb, 256 Kb, 4Mb, 64 Mb)

implementations can choose a unified or a split TLB

bits:

V - valid

L - entry locked

M - modified trap

B - branch taken trap

Caches

cache visible architecture

System Memory

Virtual Address Space

52 bit virtual address range

20 bit segment Id

32 bit segment offset

Physical Address Space

34 bit physical address range

segment Id 0 to 3 directly map to the physical address, without TLB translation and access rights checking. Only privileged mode can access these segments.

I/O Address Space

address: 0xF0000000 to 0xFFFFFFFF

always uncached

Addressing and Access Control

Address Translation

Access Control

Access Protection

Interruptions

Traps

External Interrupts

Input/Output

Concepts

memory mapped

dedicated address space in physical memory

bus concept

module concept

modules have registers - load / store, however registers do not have to be 64 bit wide.

the system bus features up to 64 modules. that is: 64 HPA pages, need: 1Mb address range.

a module listens to three address ranges

- hard physical address range. a page. privileged.

- soft physical address range. allocated by the module for further space. aligned on a page boundary.

- broadcast address range. Mirrors the system bus range. A write to a register in that range applies to all corresponding registers of a module on the same bus.

modules are processors, memory and I/O cards.

we could keep processor stats in the HPA space

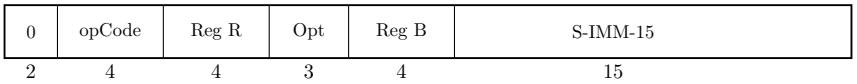
memory module has entire memory as SPA space

Instruction Set

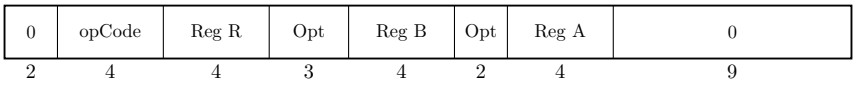
Instruction Overview

Arithmetic Instructions

The arithmetic instruction operate on two signed 64-bit operands and stores the result to the target register. There are two instruction layouts, immediate and register based. The **immediate operand instruction format** will add the sign extended 15-bit value to `RegB`. The register instruction format instruction will add `RegA` and `RegB`. The following figure shows the instruction layout.



The **register instruction format**type instruction uses two registers as input operands, performs the operation and stores the result in a third register. The following figure shows the instruction layout.



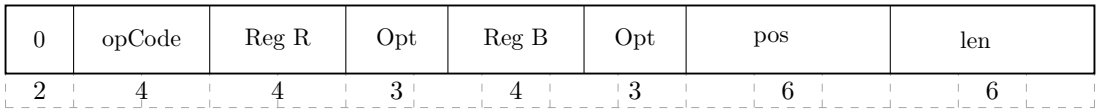
The **ADD** and **SUB** perform signed integer 64-bit arithmetic with an overflow resulting in an overflow trap. There are no operations on unsigned quantities. The **SHLxA** and **SHRxA** perform an arithmetic shift operations and add an operand to the shifted input, storing the result in the target register.

The **CMP** compares two registers for being equal, not equal, less than and les or equal than. The result of this comparison is store din the target register.

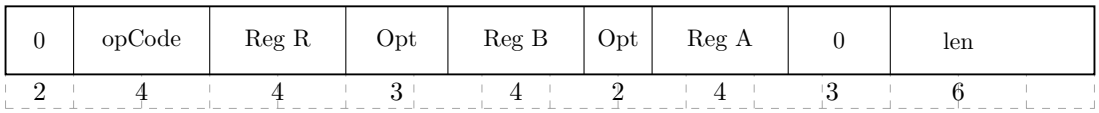
Bit Operations Instructions

The bit operation instruction fall into two groups. The **AND**, **OR** and **XOR** instructions implement the logical operations as their name suggest. Like the arithmetic instructions, there is an immediate instruction and a register instruction layout.

The second group implements the bit manipulation operations. The **EXTR** instruction extracts a bit field from a register and places the field in another register. Optionally, the extracted field is signed extended. The **DEP** instruction deposits a bit field in a target register.



The **DSR** instruction concatenates two general registers, shifts the 128-bit quantity to the right and stores the lower 32 bits in a target register.



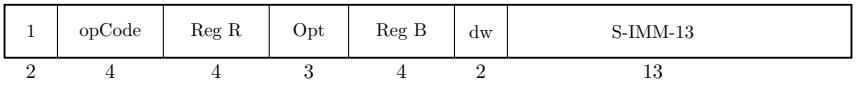
Immediate Instructions

LDI, ADDIL, LDO

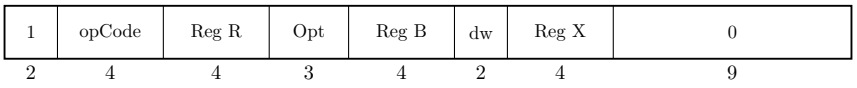
Memory Reference Instructions

T64 is a register memory architecture. It has the common load/store instructions as well as instructions that combined an operation with a memory reference. Memory reference instructions feature two addressing modes. The first is the base register and offset instruction. the second the base register and register index instruction.

The **indexed instruction format** will load the content of memory address formed by adding the scaled immediate 13-bit field to the base register `RegB` and add the data value to `RegA`. The **dw** field indicates the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. In addition, the **dw** field will scale the offset value by left shifting the immediate field according to the data width. The following figure shows the general instruction layout for base register and offset instruction.



The **register indexed instruction format** will load the content of memory address formed by adding `RegX` to the base register `RegB` and add the data value to `RegA`. Both load formats will use the **dw** field to specify the data value width. A **dw** value of zero load an 8-bit, a value of 1 a 16-bit value, value of 2 a 32-bit value and a value of 3 a 64-bit value. In any case, the data value fetched is signed extended to a 64-bit value. The offset in `RegX` is interpreted as a byte offset, independent of the actual **dw** field value. The following figure shows the general instruction layout for base register and register index type instructions.



The **LD** and **ST** instruction are the load and store instructions. In addition to the two addressing modes presented above, they also feature a base register address modification. Depending on the offset sign, the base register is updated before or after the address computation with the effective address computed. See the instruction description for more details.

The **LDR** and **STC** instruction implement the base support for a pipeline friendly method for semaphore operations. They only support the base register and offset mode. Also, there is no base register modification option.

The **ADD**, **SUB**, **AND**, **OR**, **XOR**, **CMP** instructions perform the respective operation as described above with one operand being fetched from memory.

Branch Instructions

B, BR, BV, BB, CBR, MBR

System Instructions

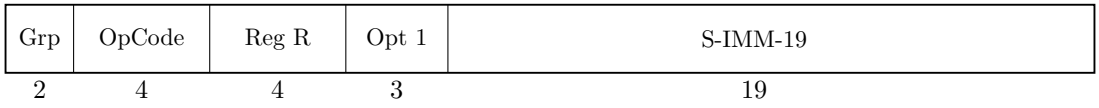
MFCR, MTCR, RSM, SSM, RFI, ITLB, PTLB, PCA, FCA, LDPA, PRBR, PRBW

Instruction Formats

T64 uses few instruction formats. They are grouped by the number of available register slots and the length of the immediate value field.

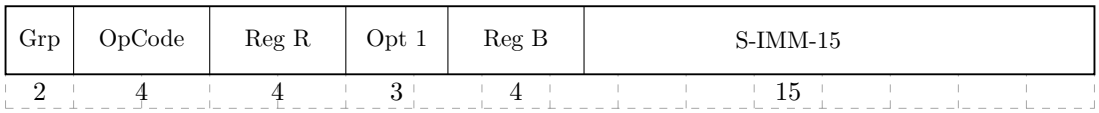
Immediate S19

The immediate S19 instruction format provides one register and a 19 bit signed immediate field. This format is typically used by branch instructions.



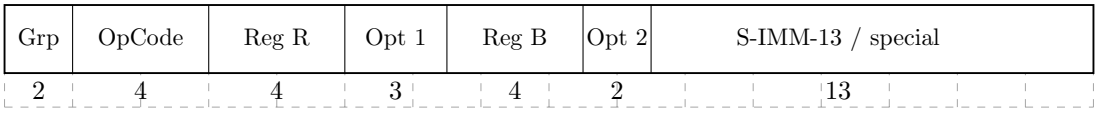
Immediate S15

The immediate S15 instruction format provides two registers and a 15 bit signed immediate field. This format is used by branch instructions as well as instruction which operate on a register and an immediate value.



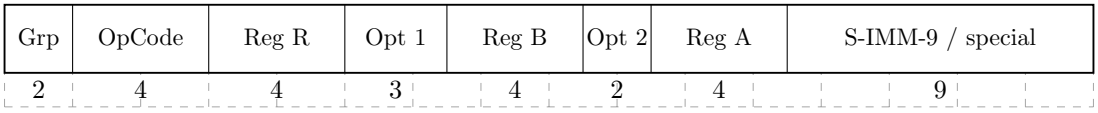
Immediate S13

The immediate S13 instruction format provides two registers, an additional option field and a 13 bit signed immediate field. Some instruction use the S-IMM-13 field for special encodings instead of a signed value. This format is used by memory reference instructions where the address is formed by a base register and a 13-bit signed offset.



Immediate S9

The immediate S9 instruction format provides three registers, an additional option field and a 9 bit signed immediate field. Some instruction use the S-IMM-9 field for special encodings instead of a signed value.instead of a signed value. This instruction format is used for all computational instructions where three registers are needed.



Immediate U20

The immediate U20 format is used by the immediate instruction group to provide a 20bit unsigned value. This value is then placed at certain positions in the register target Reg R.

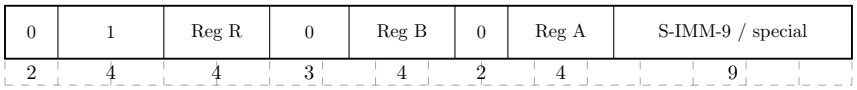
ADD Integer Addition

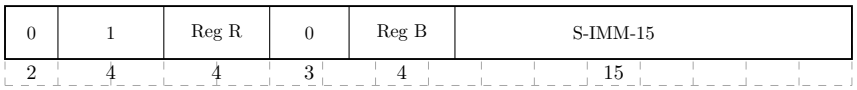
Syntax

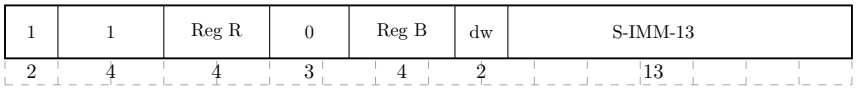
```
ADD RegR, RegB, RegA
ADD RegR, RegB, Immed15
ADD RegR, Immed13( RegB )
ADD RegR, RegX ( RegB )
```

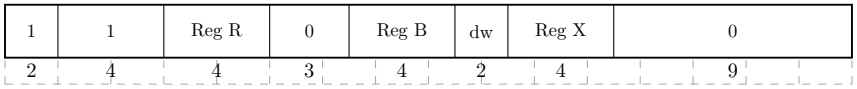
Format

The ADD instruction uses the register, the immediate, the indexed and the register indexed instruction formats.









Description

The `ADD` instruction adds two signed 64-bit operands and stores them to the target register `RegR`. The instruction supports the immediate, the register and the two memory reference instruction formats.

Operation

`RegR <- RegB + RegA` (register format)
`RegR <- RegB + immOperand( )` (immediate format)
`RegR <- RegR + memOperand( )` (indexed formats)

Exceptions

Integer Overflow
Data Alignment
Memory Access Violation
Memory Protection Violation
Data TLB miss

Notes

None.